



## Week 6 — Apache Spark Assignment

**Name:** Faizul Haque Rizwi

**Student ID:** CT\_CSI\_DE\_80

**Domain:** Data Engineering

**University:** Maharishi Markandeshwar (Deemed to be University), Mullana, Ambala, Haryana

**College:** Maharishi Markandeshwar Engineering College, Mullana, Ambala, Haryana

## Week 6 — Apache Spark Assignment

Questions & Answers (Q1 – Q15)

### Q1. Explain the roles of the Driver, Cluster Manager, and Executor in a Spark application.

#### Answer:

- Driver: The Driver is the main program that controls the Spark application. It creates the SparkSession, plans the execution, and sends tasks to the executors.
- Cluster Manager: The Cluster Manager is responsible for managing resources such as CPU and memory. It allocates resources to Spark applications.
- Executor: Executors are worker processes that execute the tasks assigned by the Driver. They perform data processing and return the results to the Driver.

### Q2. How does Spark's Lazy Evaluation strategy improve performance when chain-processing large datasets?

#### Answer:

Spark uses Lazy Evaluation, which means transformations are not executed immediately. Instead, Spark records all transformations in a Directed Acyclic Graph (DAG). The execution starts only when an action (such as `show()`, `count()`, or `collect()`) is called. This allows Spark to optimize the execution plan, reduce unnecessary computations, and improve overall performance.

### Q3. Write a Spark command to read a CSV file located at "data/source.csv", ensuring the first row is treated as a header and inferSchema is enabled.

```
df = spark.read.csv(
    "data/source.csv",
    header=True,
    inferSchema=True
)
```

#### Output:

```
Original Dataset
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|category|price|old_name|base_price|user_id|status|amount|region|priority|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|101|Electronics|1500|Phone|1200|1|Completed|1500|North|High|
|102|Furniture|500|Chair|450|2|Pending|500|South|Low|
|103|Electronics|2500|Laptop|2200|NULL|Completed|2500|North|High|
|104|Toys|300|Car|250|4|Completed|300|East|Medium|
|105|Electronics|1800|TV|1600|5|Pending|1800|West|High|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

Schema
root
 |-- product_id: integer (nullable = true)
 |-- category: string (nullable = true)
```

```
-- price: integer (nullable = true)
-- old_name: string (nullable = true)
-- base_price: integer (nullable = true)
-- user_id: integer (nullable = true)
-- status: string (nullable = true)
-- amount: integer (nullable = true)
-- region: string (nullable = true)
-- priority: string (nullable = true)
```

**Answer:**

The header=True option tells Spark that the first row of the CSV contains column names instead of data. The inferSchema=True option tells Spark to automatically detect the data type of each column (such as integer, string, or double) by scanning the file, instead of treating every column as a string by default.

**Q4. What is the difference between CSV and Parquet in terms of storage (row-based vs. columnar) and why does it matter for performance?****Answer:**

| CSV                        | Parquet                              |
|----------------------------|--------------------------------------|
| Row-based storage          | Column-based (columnar) storage      |
| Larger file size           | Smaller file size due to compression |
| Slower reading and writing | Faster reading and writing           |
| Does not store schema      | Stores schema information            |

**Why it matters:**

Because Parquet is columnar, Spark can read only the specific columns needed for a query instead of scanning the entire file row by row. This reduces disk I/O, lowers memory usage, and significantly improves query performance, especially on large datasets.

**Q5. Given a DataFrame df, write a query to select the columns product\_id and price where the category is 'Electronics'.**

```
electronics = df.filter(
    col("category") == "Electronics"
).select(
    "product_id",
    "price"
)

electronics.show()
```

**Output:**

```
Electronics Products
+-----+-----+
|product_id|price|
+-----+-----+
|      101| 1500|
|      103| 2500|
|      105| 1800|
+-----+-----+
```

**Answer:**

The filter() transformation keeps only the rows where the category column equals "Electronics". The select() transformation then narrows the DataFrame down to just the product\_id and price columns.

**Q6. Write the code to "revise" a DataFrame by renaming the column old\_name to new\_name and casting the price column from a String to a Double.**

```
df = df.withColumnRenamed("old_name", "new_name")

df = df.withColumn(
    "price",
    col("price").cast("double")
)
```

**Output:**

```
Rename old_name -> new_name
root
|-- product_id: integer (nullable = true)
|-- category: string (nullable = true)
|-- price: double (nullable = true)
|-- new_name: string (nullable = true)
|-- base_price: integer (nullable = true)
|-- user_id: integer (nullable = true)
|-- status: string (nullable = true)
|-- amount: integer (nullable = true)
|-- region: string (nullable = true)
|-- priority: string (nullable = true)
```

**Answer:**

withColumnRenamed() changes the name of an existing column without altering its data. withColumn() combined with cast("double") converts the price column's data type from String to Double, which is necessary before performing any numeric/mathematical operations on it.

**Q7. How does Spark use the Lineage Graph (DAG) to provide fault tolerance if a worker node fails?****Answer:**

Spark keeps track of all transformations applied to a dataset in a Lineage Graph (DAG). If a worker node fails and a partition of data is lost, Spark does not need to reload or recompute the entire dataset. Instead, it uses the lineage information to recompute only the lost partitions from

the original source data. This provides efficient fault tolerance without the overhead of storing multiple replicated copies of the data.

**Q8. Write a query to filter a DataFrame `df_orders` for rows where the status is 'Completed' AND the amount is greater than 1000.**

```
completed = df_orders.filter(
    (col("status") == "Completed") &
    (col("amount") > 1000)
)

completed.show()
```

**Output:**

```
Completed Orders
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|  category| price|new_name|base_price|user_id|  status|amount|region|priority|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      101|Electronics|1500.0|  Phone|    1200|      1|Completed|  1500| North|    High|
|      103|Electronics|2500.0| Laptop|    2200|    NULL|Completed|  2500| North|    High|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+
```

**Answer:**

The & operator combines two conditions so that only rows satisfying BOTH conditions are kept: status must equal "Completed" and amount must be greater than 1000.

**Q9. Explain the concept of Predicate Pushdown in Parquet and how it affects the amount of data loaded into memory.**

**Answer:**

Predicate Pushdown is an optimization technique used with the Parquet file format. When a filter condition is applied on a DataFrame, Spark pushes that filter down to the storage layer instead of loading all the data first and filtering afterward. As a result, only the rows (and, combined with columnar storage, only the columns) that satisfy the condition are actually read from disk. This greatly reduces the amount of data loaded into memory, decreases disk I/O, and improves overall query performance.

**Q10. Write a code snippet to add a new column `final_price` which is the `base_price` multiplied by 1.18 (18% tax).**

```
df = df.withColumn(
    "final_price",
    col("base_price") * 1.18
)

df.show()
```

**Output:**

```
Adding Final Price
```

| product_id | category    | price  | new_name | base_price | user_id | status    | amount | region | priority | final_price |
|------------|-------------|--------|----------|------------|---------|-----------|--------|--------|----------|-------------|
| 101        | Electronics | 1500.0 | Phone    | 1200       | 1       | Completed | 1500   | North  | High     | 1416.0      |
| 102        | Furniture   | 500.0  | Chair    | 450        | 2       | Pending   | 500    | South  | Low      | 531.0       |
| 103        | Electronics | 2500.0 | Laptop   | 2200       | NULL    | Completed | 2500   | North  | High     | 2596.0      |
| 104        | Toys        | 300.0  | Car      | 250        | 4       | Completed | 300    | East   | Medium   | 295.0       |
| 105        | Electronics | 1800.0 | TV       | 1600       | 5       | Pending   | 1800   | West   | High     | 1888.0      |

**Answer:**

withColumn() adds a new column called final\_price. Its value is computed by multiplying the existing base\_price column by 1.18, which adds 18% tax on top of the base price.

### Q11. What is the difference between Transformations and Actions? Provide two examples of each.

**Answer:****Transformations:**

- Create a new DataFrame or RDD from an existing one.
- They are lazily evaluated — not executed immediately, only recorded in the DAG.
- Examples: filter(), select()

**Actions:**

- Trigger the actual execution of all pending transformations.
- Return a result to the Driver or write data to storage.
- Examples: show(), count()

### Q12. Write the Spark command to load a Parquet file from "path/to/input", filter out any rows where user\_id is null, and save the result as a CSV at "path/to/output".

```
parquet_df = spark.read.parquet("path/to/input")

filtered = parquet_df.filter(
    col("user_id").isNotNull()
)

filtered.write.mode("overwrite").option(
    "header",
    True
).csv("path/to/output")
```

**Output:**

```

Saving as Parquet...
Reading Parquet...
Filter user_id NOT NULL
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|category|price|new_name|base_price|user_id|status|amount|region|priority|final_price|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|101|Electronics|1500.0|Phone|1200|1|Completed|1500|North|High|1416.0|
|102|Furniture|500.0|Chair|450|2|Pending|500|South|Low|531.0|
|104|Toys|300.0|Car|250|4|Completed|300|East|Medium|295.0|
|105|Electronics|1800.0|TV|1600|5|Pending|1800|West|High|1888.0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
Saving CSV

```

**Answer:**

spark.read.parquet() loads the Parquet data into a DataFrame. filter(col("user\_id").isNotNull()) removes any rows where user\_id is null. Finally, write.csv() saves the cleaned result as a CSV file, with mode("overwrite") replacing any existing output and option("header", True) writing the column names as the first row.

**Q13. In Spark Architecture, what is the difference between Client Mode and Cluster Mode?****Answer:**

| Client Mode                                                | Cluster Mode                                                  |
|------------------------------------------------------------|---------------------------------------------------------------|
| The Driver runs on the local (client) machine.             | The Driver runs inside the cluster.                           |
| Suitable for development and testing.                      | Suitable for production environments.                         |
| The client machine must remain connected during execution. | The application keeps running even if the client disconnects. |

**Q14. Write a query to filter a dataset for rows where the region is 'North' OR the priority is 'High'.**

```

north = df.filter(
    (col("region") == "North") |
    (col("priority") == "High")
)

north.show()

```

**Output:**

```

North OR High Priority
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|  category| price|new_name|base_price|user_id|  status|amount|region|priority|final_price|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      101|Electronics|1500.0|  Phone|    1200|      1|Completed|  1500| North|   High|   1416.0|
|      103|Electronics|2500.0| Laptop|    2200|    NULL|Completed|  2500| North|   High|   2596.0|
|      105|Electronics|1800.0|   TV|    1600|      5| Pending|  1800| West|   High|   1888.0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

**Answer:**

The | operator combines two conditions so that a row is kept if EITHER condition is true: region equals "North" or priority equals "High".

### Q15. When exploring a dataset, why is it safer to use .show(5) instead of .collect() on a multi-terabyte dataset?

**Output:**

```

Preview Only
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|product_id|  category| price|new_name|base_price|user_id|  status|amount|region|priority|final_price|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+
|      101|Electronics|1500.0|  Phone|    1200|      1|Completed|  1500| North|   High|   1416.0|
|      102| Furniture| 500.0|  Chair|     450|      2| Pending|   500| South|   Low|     531.0|
|      103|Electronics|2500.0| Laptop|    2200|    NULL|Completed|  2500| North|   High|   2596.0|
|      104|   Toys| 300.0|   Car|     250|      4|Completed|   300| East| Medium|     295.0|
|      105|Electronics|1800.0|   TV|    1600|      5| Pending|  1800| West|   High|   1888.0|
+-----+-----+-----+-----+-----+-----+-----+-----+-----+-----+

```

**Answer:**

.show(5) displays only the first five rows of the dataset and is memory-efficient because it processes just a small sample of partitions. .collect(), on the other hand, pulls the entire dataset from all executors into the Driver's memory. On a multi-terabyte dataset, this can easily exceed the Driver's available memory, causing an out-of-memory error or application failure. Therefore, .show(5) is the safer option for exploring large datasets.

## Program Completion

Assignment Completed Successfully